

VGADASH: Failure-Time Observability Through a VGA-Resident Kernel Dashboard

Aditya Giri, Harshita Tibra, Armaan Ahmed

CSE, Ramdeobaba University, Nagpur

giriad@rknc.edu, tibrahd@rknc.edu, ahmeda@rknc.edu

Abstract: Kernel debugging becomes difficult precisely when it is most needed. During early boot failures, graphics stack regressions, failed userspace initialization, or network outages, a Linux machine may still be executing kernel code while becoming effectively unreachable to the developer. In those states, common observability tools such as `journalctl -k`, SSH-based diagnostics, or GUI consoles are either unavailable or unreliable. VGADASH addresses this problem by providing a small in-kernel dashboard rendered directly in VGA text mode. The system is designed as a local fallback visibility path that exposes recent kernel log activity and a compact summary of system state without depending on normal userspace interaction. Its primary operator interface is the keyboard through SysRq, allowing the dashboard to be shown or navigated even when shell access is impractical [1]. This paper presents the motivation, design, and current evaluation of VGADASH as a systems-oriented semester project. The contribution is not a general observability framework, but a lightweight, recovery-focused mechanism for machines that are alive yet difficult to debug. Validation is grounded in a reproducible Docker and QEMU workflow, live demonstrations, and automated snapshot-based checks that support the visual behavior with deterministic evidence [7].

Keywords: Linux kernel module; failure-time observability; VGA text mode; SysRq; printk; QEMU

1. Introduction

Systems that fail during kernel experimentation often do not fail cleanly. A machine may stop short of a usable login shell, never complete the graphics handoff, lose remote network reachability, or hang while probing a newly modified driver. In all of these situations, the kernel may still be alive enough to execute some code, but the developer loses the normal paths used to observe what the system is doing. This mismatch creates a practical debugging problem: the system is still running, but it is no longer practically inspectable.

The usual Linux observability stack assumes a healthy dependency chain. Kernel messages are emitted through `printk`, retained in kernel-managed buffers, and later surfaced through tools such as `dmesg` or `journalctl -k` for consumption through a shell, GUI, storage device, or remote session [2]. If userspace initialization fails, if graphics is broken, or if network and storage paths are unavailable, the operator may lose access to information even though the information still exists inside the kernel. In other words, observability fails downstream of the components that are most fragile during early and low-level failures.

VGADASH is proposed as a local fallback observability mechanism for those failure-time conditions. Instead of treating visibility as a userspace service, it treats visibility as a kernel-resident capability. The system renders a small dashboard directly onto the VGA text display path and is intended to be activated through SysRq so that the operator can reach it using the keyboard and monitor alone [1]. The dashboard provides two focused views: a recent-kernel-logs page and a compact state page. This keeps the design small, direct, and aligned with the kind of information that is most useful during degraded recovery scenarios.

This paper makes three contributions. First, it frames failure-time observability as a dependency-reduction problem and motivates a local kernel-resident solution. Second, it presents the design of VGADASH as an integrated combination of VGA text rendering, in-kernel log capture, and SysRq-driven activation. Third, it evaluates the current prototype in a reproducible Docker and QEMU environment, showing that the design can be demonstrated, exercised, and regression-tested in a controlled manner [7].

2. Problem Context and Motivation

The project is motivated by a common pattern in kernel and low-level systems work: the more experimental the environment becomes, the less reliable the normal debugging path is. Early boot hangs can occur before a usable shell appears. Driver changes can break suspend and resume or hang during probe. Network-based diagnostics disappear when NIC initialization or userspace networking fails. Graphics failures can leave a machine without a usable display

stack even while the kernel continues to execute. In each case, the machine may be technically alive but operationally opaque.

This opacity is the core problem. Standard observability tools are not wrong; they are simply positioned too high in the dependency stack for the failure mode being considered. `journalctl -k`, for example, is useful under normal conditions, but it presumes healthy userspace services and a path back to the operator. Remote logging solutions shift the dependency to the network stack and a receiver [3]. Post-mortem tools such as `kdump` and `pstore` are valuable after crashes, but they serve a different diagnostic goal [4][5]. VGADASH therefore targets a narrower but important space: systems that have not fully crashed, but are no longer comfortable to debug through conventional interfaces.

From this viewpoint, the project is best understood as a dependency minimization exercise. If the purpose is to recover some visibility in degraded states, then the display path should be as primitive and local as possible, the control path should not depend on a shell, and the information path should avoid fragile userspace services. That motivation directly leads to the choices at the heart of VGADASH: VGA text mode for display, `SysRq` for control, and in-kernel capture of recent logging activity. This emphasis on preserving a small working path through failure also aligns with broader systems thinking that reduces reliance on complex recovery chains [8].

3. Background

Three background mechanisms matter for understanding the system. The first is `printk`, the Linux kernel logging interface. Kernel subsystems and drivers emit messages through `printk`, and these messages are retained in a kernel-managed buffer before being exposed to higher-level tools [2]. This means that kernel-originated information already exists inside kernel space before any userspace logger displays or persists it. That observation is the foundation of VGADASH's log design.

The second background mechanism is VGA text mode. Although modern graphics systems are far more sophisticated, VGA text mode remains a simple and direct output path on many x86-oriented virtualized environments. It is attractive for a fallback dashboard precisely because it is primitive: it requires neither a userspace compositor nor a complex rendering stack. A small amount of information can therefore be displayed in a deterministic 80x25 text layout when richer interfaces are unavailable.

The third background mechanism is `SysRq`. The Magic `SysRq` interface already exists in Linux as an emergency keyboard control path for system recovery and debugging [1]. It is not a general user interface, but that is exactly why it is useful here. `SysRq` offers a realistic way to invoke a kernel-level recovery surface with minimal dependence on normal shell interaction. In the context of VGADASH, `SysRq` is not a side feature; it is the primary operator entrypoint.

4. System Design

VGADASH combines three coordinated paths: a display path, an information path, and a control path. The display path is a non-destructive VGA text overlay. When activated, the system saves the current text buffer, renders the dashboard, and preserves a restore path so that normal console contents can be returned. This matters because the goal is not to replace the system console permanently, but to provide a temporary diagnostic surface that can be invoked and dismissed as needed.

The information path is intentionally narrow. VGADASH exposes a logs page and a state page. The logs page is meant to answer the immediate question, "what has the kernel been saying recently?" The state page is meant to answer, "what sort of machine state am I looking at right now?" This split is small enough to keep the dashboard readable in text mode, while still giving the operator two distinct types of diagnostic value: event stream visibility and quick situational context.

The control path is built around `SysRq`. The current interaction model supports a toggle key and direct page-selection keys for the logs and state pages [1]. This choice reflects the paper's core use case. If the machine is degraded, relying on a shell or filesystem-visible control node weakens the argument. `SysRq` keeps the interface close to the keyboard and closer to the kernel's own emergency control mechanisms.

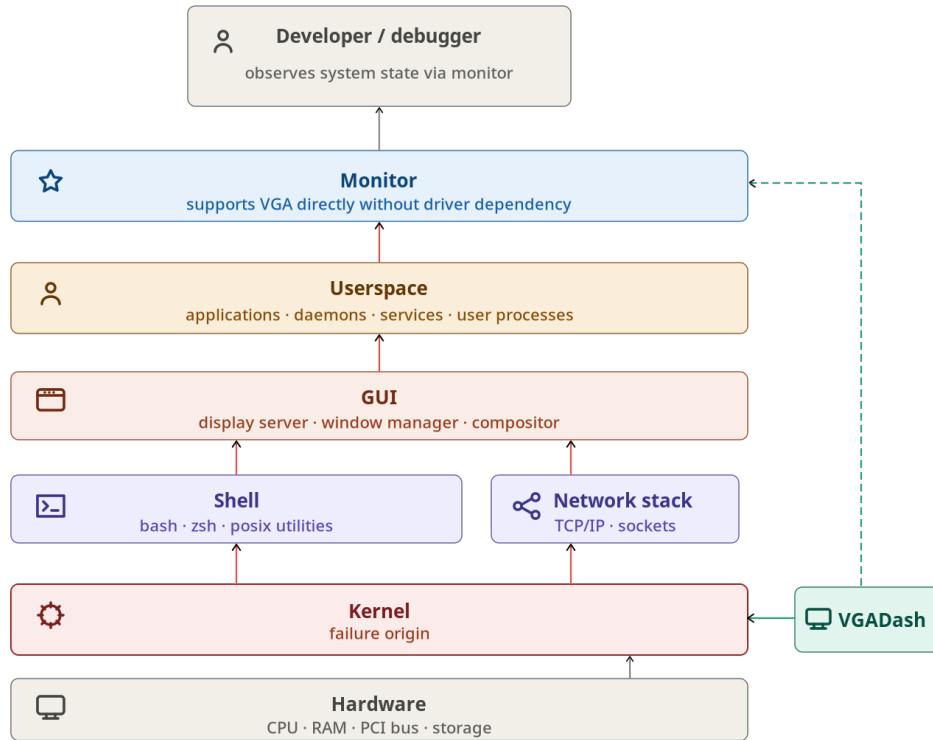


Figure 1. Failure-time observability gap and how VGADASH shortens the dependency chain.

Two design properties are especially important. First, the overlay is non-destructive. It does not claim ownership of the console forever; it temporarily uses the screen and provides a restore path. Second, the interaction model is deliberately small. VGADASH is not intended to become a general debugging shell. It is meant to preserve a thin but valuable observability layer for a machine that is no longer conveniently debuggable.

5. Key Technical Challenges

5.1 Logging Without Userspace

The first major challenge is recovering recent kernel information without depending on userspace. A naive solution would simply tell the operator to use `journalctl -k` or `dmesg`, but that misses the project's target failure scenario. Those tools are only useful when the operator can still reach them. The project therefore treats userspace logging as an unreliable downstream consumer rather than as the primary observability mechanism.

The design response is to capture kernel-originated messages while they are still inside the kernel's own logging path [2]. Rather than depending on a later userspace retrieval step, VGADASH stores recent log output in a module-owned structure that can be rendered directly to the VGA dashboard. This aligns with the presentation's key insight: the information originates in the kernel, so if it can be captured and displayed there, one major dependency chain disappears.

5.2 Activation During Degraded States

The second challenge is activation. A dashboard that requires healthy shell access to become visible is less useful in exactly the situations the paper cares about. While secondary control paths may exist for development convenience,

the real systems question is whether the operator can reach the dashboard when normal interaction has become unreliable.

The response is SysRq-driven invocation and page selection [1]. By binding the main interaction model to keyboard-triggered emergency controls, VGADASH keeps activation close to the kernel and close to the machine. This does not remove all limitations. SysRq may be disabled, keyboard interrupts may fail, and hard lockups can still prevent useful execution. Even so, the SysRq path is a better match for the target failure mode than a shell-centered interaction model.

5.3 Testing a Visual Kernel Mechanism

The third challenge is evaluation. A VGA dashboard is fundamentally visual, but a paper cannot rely on screenshots alone, and CI cannot depend on manual inspection. Without a reproducible validation strategy, the project would collapse into a demo-only artifact.

The response is to support the visual behavior with a controlled QEMU workflow and plain-text snapshots used for automated assertions [7]. The project boots the module in QEMU, injects known markers into the kernel message path, and checks that the expected page content is exported correctly. These checks do not replace the visual behavior; rather, they make it testable enough to support a technical paper with deterministic evidence instead of anecdotal screenshots.

6. Evaluation

The current evaluation is intentionally focused on operational scenarios rather than on performance microbenchmarks. The strongest claim made by this paper is that VGADASH is useful for systems that are still alive but have become difficult to inspect through normal userspace channels. This claim is supported by a controlled Docker and QEMU workflow in which the dashboard can be loaded, invoked, navigated through SysRq-equivalent paths, and validated against expected output [7].

Three results matter most. First, the logs page provides immediate visibility into recent kernel activity without relying on normal userspace log consumption as the primary narrative path. Second, SysRq-based invocation makes the dashboard reachable in a keyboard-driven recovery workflow that is much closer to the failure-time use case than a shell-first interface [1]. Third, the project's QEMU workflow supports reproducible regression checks, which makes the design more than a one-off visual demo.

The evaluation is therefore best understood as scenario validation. It shows that a machine with broken or unavailable higher-level interfaces can still expose useful information locally, that a keyboard-triggered emergency path is practical for page activation and navigation, and that the visual design can be supported by deterministic tests using exported text snapshots. Figure 2 shows the logs page reached through the SysRq path, while Figure 3 shows the complementary state page used for quick situational context.

```

VGADASH [page:logs]
-----
Last console-emitted kernel log lines (post-load):
[ 2.374638] Freeing unused kernel image (initmem) memory: 3296K
[ 2.376625] Write protecting the kernel read-only data: 30720k
[ 2.383640] Freeing unused kernel image (text/rodata gap) memory: 2036K
[ 2.386433] Freeing unused kernel image (rodata/data gap) memory: 1288K
[ 2.596296] x86/mm: Checked W+X mappings: passed, no W+X pages found.
[ 2.598278] Run /init as init process
[ 2.599001]   with arguments:
[ 2.599688]     /init
[ 2.599956]   with environment:
[ 2.600618]     HOME=/
[ 2.600738]     TERM=linux
[ 2.604718] tsc: Refined TSC clocksource calibration: 2687.536 MHz
[ 2.605504] clocksource: tsc: mask: 0xffffffffffffffff max_cycles: 0x26bd4031
[ 2.606314] clocksource: Switched to clocksource tsc
[ 2.741482] vga_dash: loading out-of-tree module taints kernel.
[ 2.744592] vga_dash: module verification failed: signature and/or required ke
[ 2.754205] printk: console [vga_dash-1] enabled
[ 2.755762] vga_dash: SysRq toggle enabled (Alt+SysRq+v)
[ 2.756399] vga_dash: SysRq logs enabled (Alt+SysRq+g)
[ 2.756776] vga_dash: SysRq state enabled (Alt+SysRq+y)
[ 2.756966] vga_dash: loaded (console-tap logs enabled)
[ 14.084731] sysrq: Show VGADASH logs page

```

Figure 2. VGADASH logs page showing recent kernel output through the VGA overlay.

```

VGADASH [page:state]
-----
Kernel: 5.15.0-174-generic
Uptime: 15 s
CPUs online: 1
Mem: total 454 MiB free 422 MiB

This CPU task: pid=17 comm=kworker/0:1

Controls:
Alt+SysRq+v toggle overlay
Alt+SysRq+g show logs page
Alt+SysRq+y show state page

```

Figure 3. VGADASH state page with compact kernel and machine context plus SysRq controls.

Packaging may be mentioned in one brief sentence as helping deployment and demonstration, but it should not appear as a result category or as a research contribution.

7. Comparison with Existing Tools

VGADASH does not replace established kernel debugging tools; it occupies a different point in the design space. SysRq offers emergency commands but does not provide an integrated visual dashboard [1]. Netconsole exports logs to the network, which is powerful when networking is healthy but fragile when the network stack is itself part of the problem [3]. Pstore and ramoops preserve crash records across resets, but they are fundamentally post-event mechanisms [4]. Kdump captures rich crash-state data at significant setup cost and is aimed at post-mortem analysis

[5]. Earlyprintk is valuable during early boot but is limited in scope and does not offer a persistent live dashboard view [6].

Related systems work also helps position the project. Crash-only software argues that recovery-oriented systems should reduce the amount of state and machinery required to regain useful operation after failures [8]. That idea aligns with VGADASH's narrow fallback scope: preserve a small but dependable visibility path instead of reconstructing a full debugging environment. Otherworld explores application survival across kernel crashes by shifting recovery into a separate operating environment [9], which is conceptually adjacent but aimed at continuity rather than direct local observability. Reverse Debugging of Kernel Failures in Deployed Systems focuses on reconstructing kernel failures after deployment with reversible execution support [10]. In contrast, VGADASH is intentionally simpler and more immediate: it surfaces live failure-time information on the local screen rather than enabling post-hoc replay or heavyweight crash recovery.

VGADASH is positioned for a narrower question: how much useful observability can be retained for a system that is still alive, but no longer comfortably accessible? Its answer is intentionally lightweight. It focuses on local monitor output, keyboard-driven control, and a unified on-screen view of recent logs and compact system state. This positioning should be emphasized as complementary rather than competitive.

Table 1. Positioning VGADASH against existing kernel debugging approaches.

Tool	Works without userspace	Needs network	On-screen visibility	Best fit
VGADASH	Yes	No	Yes	Alive-but-unusable systems needing local visibility
SysRq	Yes	No	No	Emergency commands without integrated display
netconsole	Yes	Yes	No	Remote log export when networking is healthy
pstore / ramoops	Partly	No	No	Post-reboot crash evidence
kdump	Partly	No	No	Post-mortem crash capture
earlyprintk	Yes	No	Sometimes	Very early boot logging only

8. Limitations

The present prototype has several limitations that should be stated clearly. First, the visual behavior is validated primarily in Docker and QEMU, where VGA text mode and boot behavior are controllable [7]. That makes the prototype reproducible, but it also limits the scope of the paper's claims. Second, VGA text mode is inherently environment-dependent and should not be presented as universally available across all modern Linux deployments. Third, the system is a fallback observability surface, not a replacement for rich logging, tracing, and crash-analysis infrastructure.

There are also functional limitations in the control path. SysRq is powerful, but it still depends on an operational enough system state for the relevant interrupt and deferred work path to run [1]. Hard lockups, disabled SysRq support, or platforms without a suitable VGA path can all reduce usefulness. These limitations do not invalidate the project's contribution, but they define its intended scope.

9. Future Work

The most valuable next step is broader validation. Real-hardware testing would strengthen the claims beyond QEMU, especially for display behavior and keyboard interaction. Additional platform coverage would also clarify where VGA-based fallback observability remains practical and where other low-level display paths may be needed.

Another promising direction is richer state summarization. The current dashboard deliberately stays small, but future revisions could surface more failure-oriented summaries, such as crash-adjacent subsystem health indicators or more compact driver-related status signals. Operator ergonomics can also improve through better documentation and more polished workflows for repeated recovery use.

Finally, future evaluation could add lightweight metrics around responsiveness, rendering latency, and repeated invocation cost. Such data would not redefine the project, but it would strengthen the paper by complementing the current scenario-based evidence.

10. Conclusion

VGADASH addresses a practical systems problem: the loss of observability when a Linux machine is still alive but no longer reachable through normal userspace-oriented debugging paths. By treating visibility as a kernel-resident fallback capability rather than as a userspace service, the project provides a local dashboard that remains aligned with failure-time needs.

The academic contribution of the project lies in showing that a small combination of low-level mechanisms--VGA text mode, in-kernel log capture, and SysRq-driven control--can be assembled into a useful recovery-oriented observability surface. The current implementation is intentionally limited, but it demonstrates that meaningful local visibility can be preserved under degraded conditions and validated in a reproducible environment.

References

- [1] [1] The Linux Kernel Documentation, "Linux Magic System Request Key Hacks," available at <https://docs.kernel.org/admin-guide/sysrq.html>
- [2] [2] The Linux Kernel Documentation, "Message logging with printk," available at <https://docs.kernel.org/core-api/printk-basics.html>
- [3] [3] The Linux Kernel Documentation, "Netconsole," available at <https://docs.kernel.org/networking/netconsole.html>
- [4] [4] The Linux Kernel Documentation, "Ramoops oops/panic logger," available at <https://docs.kernel.org/admin-guide/ramoops.html>
- [5] [5] The Linux Kernel Documentation, "Documentation for Kdump - The kexec-based Crash Dumping Solution," available at <https://docs.kernel.org/admin-guide/kdump/kdump.html>
- [6] [6] The Linux Kernel Documentation, "The kernel's command-line parameters," earlyprintk section, available at <https://docs.kernel.org/6.13/admin-guide/kernel-parameters.html>
- [7] [7] QEMU Documentation, "System Emulation," available at <https://www.qemu.org/docs/master/system/index.html>
- [8] [8] George Candea and Armando Fox, "Crash-Only Software," Proceedings of the 9th Workshop on Hot Topics in Operating Systems (HotOS IX), 2003.
- [9] [9] Alex Depoutovitch and Michael Stumm, "Otherworld: Giving Applications a Chance to Survive OS Kernel Crashes," Proceedings of the 14th ACM SIGOPS International Workshop on Hot Topics in Operating Systems, 2013, available at <https://www.usenix.org/conference/hotos13/workshop-program/presentation/depoutovitch>
- [10] [10] Xinyang Ge, Haowei Wu, Frank Mueller, and Xiangyu Zhang, "Reverse Debugging of Kernel Failures in Deployed Systems," 2020 USENIX Annual Technical Conference (USENIX ATC 20), available at <https://www.usenix.org/conference/atc20/presentation/ge>